CSD493 PROJECT-1 · SECURITY RESEARCH STUDY

Behavioural-Diff Detection of Malicious Browser-Extension Updates

Feasibility, Security Gap & Novelty — an Evidence-Based Study

Team: Bind Pratap Singh (2310110084) · Suhani Deepak Agrawat (2310110717)

Advisor: Dr. Sweta Mishra · **Specialisation:** Cybersecurity

Programme: B.Tech CSE · Shiv Nadar Institution of Eminence

Prepared: September 2026 · **Project:** extdrift

> **Scope.** This study answers, from peer-reviewed literature and vendor documentation rather than from our own optimism, the three questions raised by the advisor: (1) does an extension's language/runtime affect its security, (2) can our system realistically protect an end user, and (3) how do we differ from existing browser/store defences. It was written to *test*, not assume, the project's novelty and feasibility — and two findings reshape our positioning.

Contents

0 Two corrections this research forces on our own materials

1 Executive Summary

2 Extension Security Architecture

3 Language / Runtime Security (Q1)

4 Existing Browser / Store Defences (Q3)

5 Real-World Incidents — where security actually failed

6 End-User Feasibility (Q2)

7 Research Gap & Novelty

8 Capability Comparison Table

9 Recommended (Narrowed) Research Problem

10 Suggested Experiment to Prove Added Value

11 Our Prototype — current output

12 What We Should Tell Our Professor

13 References

FACT established / vendor-documented **FINDING** from a cited paper/report **ASSUMPTION** reasonable, unverified

OURS our interpretation/design

Figures read at search-level from papers are marked *verify in source* — confirm the exact number in the PDF before quoting it to an examiner.

0. Two corrections this research forces on our own materials

> **⚠ 1 — The core idea is partly taken; we must reposition, not retreat.**
> A top-tier paper already does *update-delta* detection of malicious extensions: **Pantelaios, Nikiforakis, Kapravelos, "You've Changed: Detecting Malicious Browser Extensions through their Update Deltas," ACM CCS 2020** (DOI · PDF). It analyses **922,684 versions over six years** and finds **143 malicious extensions in 21 clusters** *(verify in source)*. **But it is STATIC code-delta analysis** — it locates the *code* added between versions and encodes the delta by the APIs it abuses; its first stage even triggers on anomalous *ratings*. It is admitted to be **obfuscation-evadable**. So "diff two versions" is **not** our novelty — our wedge is "diff two versions **dynamically, by behaviour, with deterministic inputs**," which is obfuscation-resistant by construction (see §7).

> **⚠ 2 — Our "Karami (NDSS 2021)" citation is misattributed; fix it before the meeting.**
> The real paper — **Karami, Ilia, Polakis, "Awakening the Web's Sleeper Agents: Misusing Service Workers for Privacy Leakage," NDSS 2021** — is about the **Service Worker *web* API abused by *websites***, **not** Manifest V3 *extension* background service workers. Citing it for "MV3 extension service workers leak without a page" is a mischaracterisation an examiner could catch. Recharacterise it accurately, or cite the Chrome MV3 architecture docs plus a MV3-security paper (§3). Karami also authored *Carnus* (NDSS 2020) on extension *fingerprinting* — do not conflate the two.

Neither is fatal. Both make us look **more** rigorous if *we* raise them first.

1. Executive Summary

- FINDING **Extension insecurity is not a language/runtime problem.** Extensions are JavaScript/HTML/CSS in a memory-safe engine; the dominant risks are **permissions, host access, powerful browser APIs, third-party/supply-chain code, and human/account compromise** — not memory bugs or the choice of JS vs WASM. *Implication: the "language-aware analysis" angle in our original pitch is weak and should be dropped.* (§3)
- FINDING **For an ordinary end user, prevention-before-execution is largely infeasible for an external tool.** Updates are silent and automatic; the browser gives outside tools no pre-execution veto. Realistic value is at the **store / enterprise / researcher** tier and as **fast detection + response**, not consumer prevention. (§6)
- FACT **Browsers/stores already do a lot** — package signing, manifest/permission validation, automated static + manual review (updates included), malware/reputation, isolation/CSP, and MV3's remote-code ban. FINDING **But these are bypassable** by obfuscation, dormant payloads, staged rollouts, and — above all — **compromised-legitimate extensions** whose malicious *update* passes review. Cyberhaven (Dec 2024) was an **OAuth/account compromise**, not an extension-analysis failure. (§4–§5)
- OURS **The honest, defensible gap:** *deterministic, dynamic, behavioural diffing of consecutive versions to catch security-relevant behavioural change introduced by an update — especially in compromised-legitimate extensions — with explainable output.* The static version (You've Changed) exists; the dynamic, determinism-anchored, obfuscation-resistant version is open. (§7)

2. Extension Security Architecture (dev → store → install → runtime)

FACT **Build & languages.** Modern extensions (Chrome/Chromium, Edge, Firefox, Safari) are **web apps**: JavaScript/TypeScript, HTML, CSS, JSON manifest; optionally WebAssembly and — via **native messaging** — a separately-installed native host. They run as **content scripts** (injected into pages), a **background service worker** (MV3) or page (MV2), and optional UI pages. Capabilities are gated by declared **permissions** and **host permissions** (Chrome MV3 docs).

FACT **Distribution & update.** Extensions ship through a **store** (Chrome Web Store, Firefox AMO, Edge Add-ons, Safari via an Xcode-built app). The browser **auto-updates** them silently; the user is not asked to re-approve unchanged permissions, and some permission additions raise no clear prompt — **"silent permission creep"** (Pluto Security).

FACT **Store review (Chrome).** Every submission — **including updates** — goes through **automated analysis then human review**: malware-signature scanning, obfuscation and dangerous-API heuristics, policy checks, manifest validity, install testing (review-process docs).

FACT **Runtime model.** Content scripts run in an **isolated world**; the extension is constrained by **CSP** and its permission set; **MV3 forbids remotely-hosted code** (improve-security docs). Once installed, an extension with broad host access effectively **sits in the middle of the user's web traffic** (Island).

3. Language / Runtime Security — does the language matter? (Q1)

Short answer: FINDING **No, not materially.** The evidence points to *permissions, APIs, logic, dependencies, and human/supply-chain factors* as the real sources of risk — not the language or runtime.

- FACT **Memory safety is a non-issue.** Extension code runs in the browser's memory-safe JS engine; classic C/C++ bugs (buffer overflow, use-after-free) do not arise from extension JS. That whole vulnerability class is absent by construction.
- FINDING **The real vectors are capability- and logic-level:** over-broad host permissions, abuse of powerful APIs (`cookies` , `webRequest` / `declarativeNetRequest` , `scripting` , `tabs`), insecure message passing, DOM injection, and **dynamic code execution**, plus **third-party/supply-chain** code (Island, Aikido).
- FINDING **JS-specific weaknesses (e.g., prototype pollution) are logic bugs, not runtime faults** — they come from unsafe object handling in code/deps.
- FINDING **WebAssembly / native components add analysis *difficulty*, not a dominant attack class.** WASM and obfuscation mainly make code harder to inspect *statically*; native messaging widens the trust boundary. Their real relevance to us: they **defeat static analysis** — an argument *for* dynamic analysis, not for "language-aware" analysis.
- FINDING **MV3 reduced the runtime surface** — removed remote code, deprecated **~87.8% of vulnerable APIs** *(verify in source:* Manifest V3 Unveiled, arXiv 2404.08310*)* — yet **~29.8% of analysed malicious extensions remained operational after V3 conversion** and **~56% adapted** via web-accessible resources *(verify in source)*. The remote-code ban is **circumventable** by shipping configuration/rules instead of code (Palant, 2025).

> **Consequence for difficulty & design** OURS
>
> Because risk lives in *behaviour and capability*, not in the language, a reliable analyser must observe **what the extension does at runtime with its permissions**. This is *why* static analysis is hard (obfuscation/WASM/dynamic code hide intent) and *why* our dynamic, behavioural stance is the right lever. It also means we should **not** claim "language-aware detection" as a contribution — the evidence doesn't support it.

4. Existing Browser / Store Defences — what they catch, where they stop (Q3)

Mechanism	What it does	Where it stops (FINDING)
Package signing / CRX validation	Integrity + provenance of the package	Doesn't judge **behaviour**; a signed update from a **compromised** account is still trusted
Manifest & permission validation	Enforces declared capabilities	Declaration ≠ intent; **silent permission creep** on update is often unprompted
Automated static review	Malware signatures, obfuscation & dangerous-API heuristics	**Bypassable** by obfuscation, WASM, dynamic code, **dormant/condition-gated payloads**
Human review	Manual code/policy check	Not exhaustive; can't run every trigger path; **post-review updates** & staged rollouts evade
Malware/reputation + blocklist	Flags known-bad, revokes	**Reactive** — needs prior knowledge; zero-day updates slip until reported
Isolation / CSP / MV3 no-remote-code	Shrinks runtime surface	**~30% of malicious extensions still function post-MV3**; rules/config channels evade
Store review applies to updates	Same pipeline for updates	Same weaknesses; **trusted-then-malicious** update is the blind spot

FINDING **Cross-browser differences. Firefox AMO** signs every add-on after automated validation, with manual review at any time (Extension Workshop); a long-standing critique is that automated scanning is limited (Stillman, 2015). **Safari** ships extensions **inside a signed macOS/iOS app built with Xcode** — a different artifact, toolchain and review queue, and a macOS-only, structurally different target (why we exclude it).

> *The pattern: existing defences are strong on **integrity, declaration, and known-bad**, and weak on **novel behavioural change on update** — especially obfuscated/dormant payloads and compromised-legitimate extensions.*

5. Real-World Incidents — where exactly did security fail?

Incident	What happened	Failure locus (FINDING)	Detected
Cyberhaven + campaign (Dec 2024)	OAuth-phishing of dev accounts ("Privacy Policy Extensions" app) → malicious updates to **~35 extensions / 2.6M users** exfiltrating cookies/session tokens	**Account/OAuth compromise + supply-chain**, *not* an extension-analysis failure; the update then shipped through the normal pipeline	**After** exposure — live ~24h+
The Great Suspender (2021)	Extension (2M+ users) **sold to unknown party**, which added tracking/remote-code	**Ownership transfer + trust hand-off**; malicious code arrived via update	After; later removed
Nano Adblocker/Defender (2020)	Devs **sold rights**; new owner added data-collection	**Ownership transfer / acquire-and-subvert**	After user reports
DataSpii (2019)	Several extensions exfiltrated browsing/PII at scale	**Permission abuse + undisclosed collection**	After investigation

Sources: Sekoia, SecurityWeek, The Register, LWN.

> **The distinction the professor drew is correct and central** OURS
> *Developer-account/OAuth compromise ≠ extension-analysis failure*. In Cyberhaven no analyser was defeated at upload time — the **account** was taken and the update **passed review**. This is exactly the case a behavioural *update-diff* addresses: it doesn't care *how* the bad code got in (phishing, sale, insider); it asks **"does this new version suddenly *do* something the trusted one didn't?"** — the one question that catches trusted-then-malicious regardless of entry vector.

6. End-User Feasibility — protect the user before the update runs? (Q2)

> **Honest conclusion (OURS)**
> For an ordinary end user, an external tool generally **cannot prevent** a malicious update from executing — it can, at best, **detect fast and respond**. Real prevention lives one tier up (store / enterprise / researcher).

- FACT **Updates are silent and automatic.** The user never sees the new version before it runs; there is no natural interception point for a third party on a consumer machine.
- FACT **The browser gives outside tools little pre-execution power.** An extension can't reliably inspect or veto a *sibling* extension's update; an OS agent sees files but not browser-internal behaviour without deep, privileged instrumentation.

Deployment model	Pre-exec prevention?	Visibility	Notes
Browser-integrated (vendor)	Yes	Full	Only the vendor can do this; not us
Store-side scanning (pre-distribution)	**Yes** — before users get it	Full (all versions)	The natural home for update-diffing; "perfect fidelity"
Enterprise policy / fleet monitoring	Partial (allowlist + fast pull)	High	Realistic, valuable, deployable

Deployment model	Pre-exec prevention?	Visibility	Notes
Endpoint / EDR agent	Detect > prevent	Medium	Sees files/network, not browser internals
A security *extension*	No (can't veto siblings)	Low	Weakest option; avoid claiming consumer prevention
Researcher/analyst tool (ours today)	Detect (offline)	Full (lab)	Where our prototype sits; strongest for evidence

> *Say plainly to the professor: "We can't stand between Google's silent auto-update and one user's browser; we can give whoever governs the update channel — a store reviewer, an enterprise, a researcher — an automatic behavioural re-check of every new version, which is exactly the check nobody runs today."*

7. Research Gap & Novelty — classified honestly

Solved well FACT **:** package signing/integrity; permission declaration & isolation; CSP; MV3 remote-code ban; known-malware/reputation blocklisting; store review pipeline.

Partially solved FINDING **:** static review of updates (bypassable); permission-change surfacing (silent creep); **static update-delta detection** — solved by *You've Changed* (CCS 2020), but statically and thus obfuscation-limited.

Weakly solved FINDING **:** scalable **dynamic behavioural** analysis of extensions (Hulk 2014, ExtPrivA S&P 2023, Arcanum USENIX 2024, Fakeium 2024 exist, but are **single-version**); detecting **dormant/condition-gated** payloads; explainable verdicts.

Open / unsolved OURS **: dynamic, behavioural, cross-version diffing with record/replay determinism** — scoring the behavioural *change* of an update obfuscation-resistantly, targeting **compromised-legitimate** extensions, with **explainable** output. No work we found occupies this exact cell.

> **The novelty wedge, stated defensibly**
> Prior update-aware detection (You've Changed, CCS 2020) is **static code-delta** and, by the authors' own account, evadable by obfuscation. Prior dynamic behavioural analysis (Hulk, ExtPrivA, Arcanum) is **single-version**. **extdrift sits at the intersection: behavioural update-diffing made reliable by record/replay determinism** — obfuscation-resistant (we observe effects, not code) and false-positive-resistant (both versions see byte-identical input, so the diff is the extension, not web noise).

OURS **What to drop from the original idea:** "language-aware analysis" (§3 shows it doesn't matter) and any claim of **consumer pre-execution prevention** (§6 shows it's infeasible for an external tool).

8. Capability Comparison Table

"~" = partial; claims limited to what evidence supports.

Capability	Chrome/Chromium	Firefox	Edge	Safari	extdrift (ours)
Static / package checks	Yes	Yes	Yes	Yes (app-signed)	~ (manifest/perm diff)

Capability	Chrome/Chromium	Firefox	Edge	Safari	extdrift (ours)
Permission checks	Yes	Yes	Yes	Yes	Yes (+ escalation diff)
Malware / reputation	Yes	Yes	Yes	Yes	No (not our aim)
Update validation	Yes (integrity)	Yes	Yes	Yes	**Yes — behavioural**
Runtime protection (isolation/CSP/MV3)	Yes	Yes	Yes	Yes	No (analyse, not enforce)
Pre-installation protection	Yes (store)	Yes (store)	Yes	Yes	~ (if store/enterprise-deployed)
Runtime behavioural detection	~ limited	~	~	~	**Yes**
Update-to-update *behavioural* comparison	**No**	**No**	**No**	**No**	**Yes (our core)**
Protection from **compromised-legit** extensions	~ reactive	~	~	~	**Yes (design target)**
Obfuscation-resistant update analysis	~ (static, evadable)	~	~	~	**Yes (dynamic)**
Explainable per-verdict output	No	No	No	No	**Yes (rules + SHAP)**

Static code-delta update analysis (*You've Changed*) is a research method, not a browser feature, so it isn't a column; the row that matters against it is **obfuscation-resistant update analysis**, where our dynamic approach is the differentiator.

9. Recommended (Narrowed) Research Problem

> *Given two consecutive versions of the same browser extension, detect — dynamically and obfuscation-resistantly — whether the update introduces security-relevant behavioural change (new data access, new exfiltration, new capability use), with an explainable verdict, using record/replay determinism to suppress false positives; targeted at compromised-legitimate extensions and positioned for store-side / enterprise / researcher deployment rather than consumer pre-execution prevention.*

Why this is the strongest version: it (a) **concedes** the prior static update-delta work and claims only the open dynamic/deterministic cell; (b) **drops** the two indefensible claims (language-aware; consumer prevention); (c) foregrounds **determinism + explainability**, which address the field's real bottleneck (false positives) rather than chasing model accuracy.

10. Suggested Experiment to Prove Added Value

> **The one experiment that proves we add something over the closest prior work**
> **Head-to-head on obfuscation.** Take update pairs where the malicious `v2` carries an **obfuscated / dynamically-decoded / condition-gated** payload. Show that a **static code-delta** approach (You've-Changed style) **misses or mis-scores** it, while our **dynamic behavioural diff** still flags it because the payload's *runtime effect* (cookie read → new-host POST from the service worker) is observable regardless of how the code is written.

Plus two supporting results we can already produce:

1. **False-positive rate on real benign updates** (GitHub-release pairs of open-source extensions) — the metric that actually matters — **with vs. without record/replay determinism** (the ablation), showing determinism drives false positives toward zero.
2. **Detection + ground truth** on the synthetic weaponisation corpus and the reconstructed **Cyberhaven** pair, each with an explainable verdict.

11. Our Prototype — current output

The pipeline already runs end-to-end on a real browser (loads two versions, drives an identical scripted session against deterministic pages, records four channels — network / DOM / storage / API incl. the service worker — diffs, scores, and explains). **~3,100 lines of Python + ~300 of instrumentation JS; 72 automated tests, all green** (2 launch a real Chromium). Representative output:

Weaponised update (Cyberhaven-style payload actually executes)	`MALICIOUS · 0.99 · leak confirmed`
Gray-zone update (scraper batches contacts to a new host)	`SUSPICIOUS · 0.68`
Benign feature update (reading-time badge)	`BENIGN · 0.25 · no leak`

The verdict is cross-checked against a fake collector's own log (independent ground truth of what actually left the browser). Reproduce: `python scripts/demo.py` (live) or `python scripts/demo_offline.py` (no browser).

12. What We Should Tell Our Professor

1. **Why is this problem difficult?** Malice is **behavioural and can be hidden** — obfuscated, dynamically decoded, or condition-gated — so static inspection is evadable and dynamic inspection must *trigger* the behaviour and separate it from web noise. Scarce labelled data and silent auto-updates make it harder still.
2. **What makes extension security technically challenging?** Extensions are **trusted, auto-updating code with mid-traffic privilege**; trust is granted once and never re-checked; the malicious change usually arrives **after** all the checks, via an update.
3. **Does language/runtime choice materially matter? No.** Extensions are memory-safe JS; risk comes from permissions, APIs, logic, dependencies, and account/supply-chain compromise. (So we drop the "language-aware" angle.)
4. **Can the end user be protected before a malicious update executes? Not by an external tool, realistically.** Prevention belongs to the vendor/store; our value is **store/enterprise/researcher re-verification** and **fast detection**.
5. **What do browsers already check?** Signing/integrity, manifest/permissions, automated + manual review (updates included), malware/reputation, isolation/CSP, MV3 no-remote-code.
6. **Where do those defences actually fail?** Obfuscated/dormant payloads, silent permission creep, and **compromised-legitimate extensions** whose malicious update passes review — as in Cyberhaven, an **OAuth/account** compromise, not an analysis failure.
7. **How can our approach be different? Dynamic, behavioural, cross-version diffing with record/replay determinism** — obfuscation-resistant and false-positive-resistant, with explainable verdicts. The open cell next to static update-delta (You've Changed) and single-version dynamic work (Hulk, ExtPrivA).

8. **What should we change/narrow?** Drop language-aware analysis and consumer pre-execution prevention; reposition as a store/enterprise/researcher method; foreground determinism + explainability; and **cite You've Changed explicitly** as prior work we improve on dynamically.
9. **Strongest research problem to pursue?** The narrowed statement in §9.
10. **What experiment proves we add value?** The **obfuscation head-to-head** (§10) plus the **determinism ablation** on real benign updates.

13. References

Closest prior work / core papers

Pantelaios, Nikiforakis, Kapravelos. *You've Changed: Detecting Malicious Browser Extensions through their Update Deltas*. ACM CCS 2020. dl.acm.org/doi/10.1145/3372297.3423343 · PDF

Bui, Tang, Shin. *Detection of Inconsistencies in Privacy Practices of Browser Extensions (ExtPrivA)*. IEEE S&P 2023.

Kapravelos, Grier, Chachra, Kruegel, Vigna, Paxson. *Hulk: Eliciting Malicious Behavior in Browser Extensions*. USENIX Security 2014.

Bandhakavi, King, Madhusudan, Winslett. *VEX: Vetting Browser Extensions for Security Vulnerabilities*. USENIX Security 2010.

Karami, Ilia, Polakis. *Awakening the Web's Sleeper Agents: Misusing Service Workers for Privacy Leakage*. NDSS 2021. (Web service workers, not extension MV3 — see §0.)

Karami, Ilia, Solomos, Polakis. *Carnus: Exploring the Privacy Threats of Browser Extension Fingerprinting*. NDSS 2020. PDF

Platform / MV3 / defences

Chrome for Developers. *What is Manifest V3 · Improve extension security · Review process*. what-is-mv3 · improve-security · review-process

Manifest V3 Unveiled: Navigating the New Era of Browser Extensions. arXiv 2404.08310, 2024. arxiv.org/pdf/2404.08310

Palant. *Malicious extensions circumvent Google's remote code ban*. 2025. palant.info

Mozilla. *Add-ons signing & distribution*. extensionworkshop.com · Stillman, *Firefox Extension Scanning is Security Theater*, 2015. danstillman.com

Recent dynamic / analysis work

Khandelwal, Nayak, Fernandes, Fawaz. *Experimental Security Analysis of Sensitive Data Access by Browser Extensions*. ACM WWW 2024. dl.acm.org/doi/10.1145/3589334.3645683

Xie et al. *Arcanum: Detecting and Evaluating the Privacy Risks of Browser Extensions*. USENIX Security 2024. PDF

CodeX: Contextual Flow Tracking for Browser Extensions. CODASPY 2025. dl.acm.org/doi/10.1145/3714393.3726495

Fakeium: A Dynamic Execution Environment for JavaScript Program Analysis. arXiv 2410.20862, 2024. arxiv.org/pdf/2410.20862

A Study on Malicious Browser Extensions in 2025. arXiv 2503.04292. arxiv.org/pdf/2503.04292

Incidents / supply chain

Sekoia. *Targeted supply chain attack against Chrome browser extensions*. 2025. sekoia.io

SecurityWeek. *Cyberhaven Chrome Extension Hack Linked to Widening Supply Chain Campaign*. securityweek.com

The Register. *The Great Suspender*. 2021. theregister.com

LWN. *Malware in open-source web extensions (Nano Adblocker)*. lwn.net/Articles/846272

Pluto Security. *Chrome Extension Supply Chain Attacks: silent permission creep*. pluto.security

---

Prepared for the CSD493 progress meeting · extdrift · This study was written to test, not assume, the project's novelty and feasibility. Tag key: FACT = established/vendor-documented · FINDING = from a cited source · ASSUMPTION = reasonable but unverified ·

OURS = our interpretation.